第 7 章：隐私计算简介

赵俊舟

西安交通大学网安学院

junzhou.zhao@xjtu.edu.cn

2026 年 4 月 23 日

目录

1. 密码学承诺
2. 秘密共享
3. 安全多方计算

目录

1 密码学承诺
- 基本概念
- 几种承诺

2 秘密共享

3 安全多方计算

目录

1 密码学承诺
- 基本概念
- 几种承诺

2 秘密共享

3 安全多方计算

猜拳与分歧终端机

密码学承诺的应用场景

例（在线拍卖）

在拍卖中，竞标者在拍卖开始时使用承诺来提交他们的出价，而不透露具体的出价金额。拍卖结束时，所有出价可以被揭示并验证，以确保竞标者的出价是诚实的。

例（电子投票）

在电子投票系统中，承诺可以确保选民在投票时能够保密其选择，同时在投票结束后能够验证其投票的有效性。

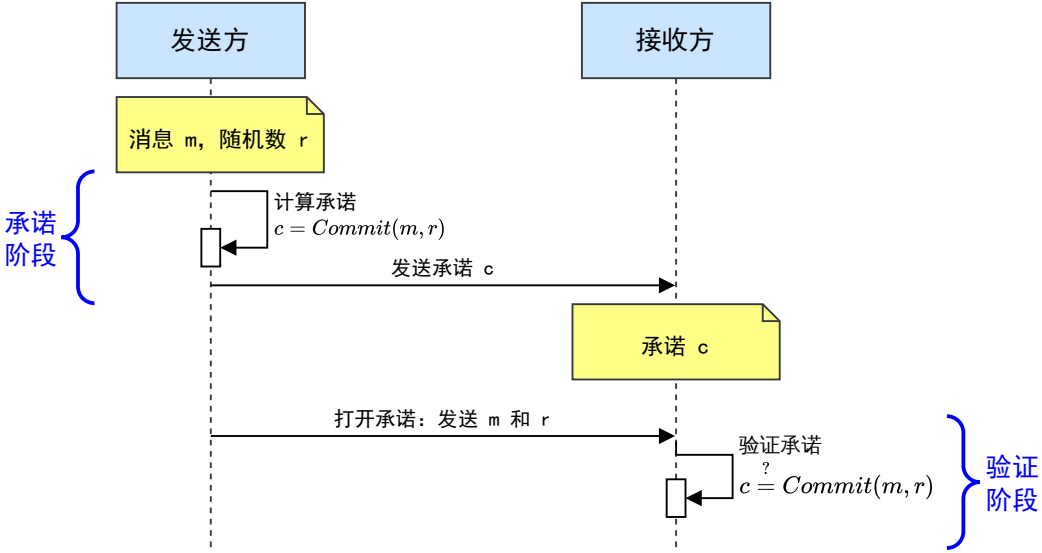
例（加密货币）

在区块链技术中，承诺可以用于确保交易的隐私和安全性。例如，某些隐私币（如 Zcash）使用承诺来隐藏交易金额和发送者信息。

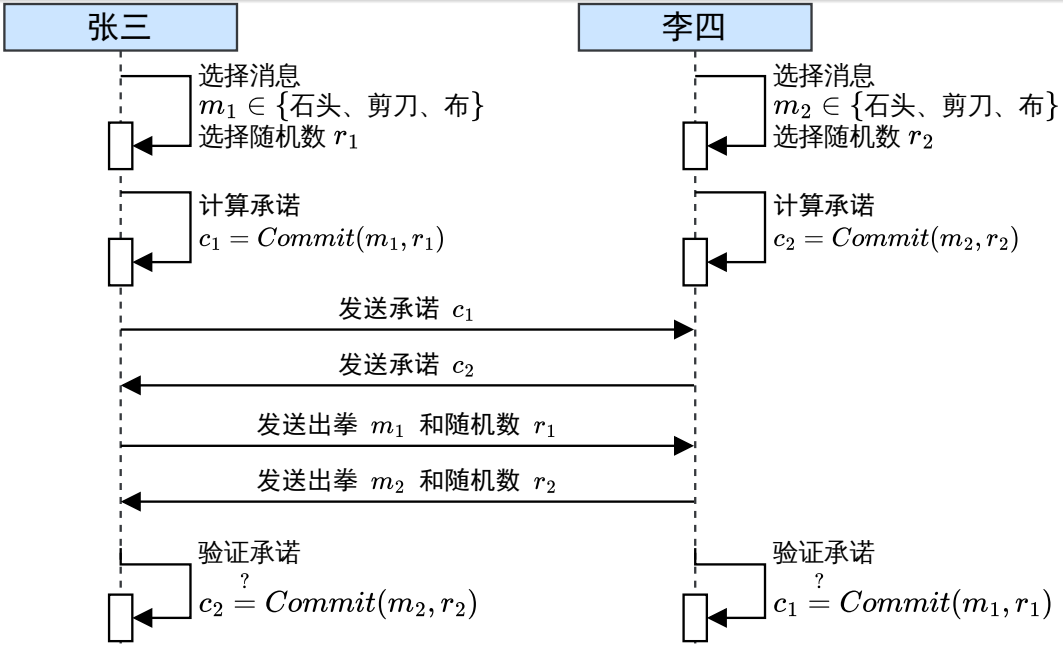
密码学承诺的基本概念

- 承诺（Commitment Scheme）是一个重要的密码学原语（Cryptographic Primitive）。承诺是一种加密协议，允许发送者承诺一个选择的值（或声明），同时对接收者保持隐藏，而接收者能够在稍后验证所承诺的值。
- 承诺通常可以分为两个阶段。
  - 承诺阶段: 发送方发送一个承诺值给接收方，这个值是发送方选择的，接收方无法知道这个值的内容。
  - 打开阶段: 发送方打开这个承诺，接收方可以验证这个值的内容。
- 密码学承诺具有两个属性:
  - 隐藏性（Hiding）：接收方无法知道发送方所承诺的值。
  - 绑定性（Binding）：发送方无法修改承诺值对应的明文。

密码学承诺的基本流程


发送方
接收方
消息 m，随机数 r
承诺
阶段
计算承诺
$c = Commit(m, r)$
发送承诺 c
承诺 c
打开承诺：发送 m 和 r
验证承诺
$c \stackrel{?}{=} Commit(m, r)$
验证
阶段

举例：公平的远程猜拳


张三
李四
选择消息
$m_1 \in \{$石头、剪刀、布$\}$
选择随机数 r_1
选择消息
$m_2 \in \{$石头、剪刀、布$\}$
选择随机数 r_2
计算承诺
$c_1 = Commit(m_1, r_1)$
计算承诺
$c_2 = Commit(m_2, r_2)$
发送承诺 c_1
发送承诺 c_2
发送出拳 m_1 和随机数 r_1
发送出拳 m_2 和随机数 r_2
验证承诺
$c_2 \stackrel{?}{=} Commit(m_2, r_2)$
验证承诺
$c_1 \stackrel{?}{=} Commit(m_1, r_1)$

目录

1 密码学承诺
- 基本概念
- 几种承诺

2 秘密共享

3 安全多方计算

哈希承诺

- 哈希承诺是密码学承诺中最简单的一种实现方式。
- 哈希承诺通过以下公式计算关于敏感数据 m 的承诺：

$$c = H(m \| r)$$

  其中 H 是密码学安全哈希函数，r 是一个随机数。
- 隐藏性：基于哈希函数的单向性，难以通过哈希值反推出敏感数据 m；
- 绑定性：基于哈希函数的抗碰撞性，难以找到不同的敏感数据 m' 产生相同的哈希值。

Pedersen 承诺

- Pedersen 承诺是目前隐私保护方案中使用广泛的密码学承诺。
- 设 $\mathbb{G}_q$ 是 $\mathbb{Z}_p^*$ 的阶为 q 的子群，g, h 为 $\mathbb{G}_q$ 的生成元。
- 承诺阶段：发送方选择一个明文 m 和一个随机数 r，计算承诺值 $C = g^m h^r \bmod p$，并发送 C 给接收方。
- 打开阶段：发送方揭示明文 m 和随机数 r。
- 验证阶段：接收方重新计算承诺值 $C' = g^m h^r \bmod p$，并验证 C' 和 C 是否相等。
- Pedersen 承诺的隐藏性和绑定性是基于 DLP 问题的困难性：
  - 隐藏性：接收方无法从承诺值 C 推导出明文 m；
  - 绑定性：发送方无法找到两个不同的 (r_1, m_1) 和 (r_2, m_2)，使得 $C = g^{m_1} h^{r_1} = g^{m_2} h^{r_2} \pmod p$。

Pedersen 承诺的绑定性

- 假设发送方找到两个不同的 (r_1, m_1) 和 (r_2, m_2)，使得 $C = g^{m_1} h^{r_1} = g^{m_2} h^{r_2} \pmod p$
- 则有
$$g^{m_1} h^{r_1} = g^{m_2} h^{r_2} \Rightarrow g^{m_1 - m_2} \equiv h^{r_2 - r_1} \pmod p$$
- 由于 g 和 h 是独立生成元，即它们生成的子群没有重叠，这意味着只有在 $m_1 - m_2 = 0$ 和 $r_2 - r_1 = 0$ 时才成立，即：$m_1 = m_2, r_1 = r_2$。
- 与假设矛盾，因此 Pedersen 承诺具有绑定性。

Pedersen 承诺

- Pedersen 承诺是目前隐私保护方案中使用广泛的密码学承诺。
- 相比哈希承诺，构造略微复杂，但提供了一系列优异的特性：
  - 信息论安全的理论最强隐藏性；
  - 基于离散对数困难问题的强绑定性；
  - 具有同态加法特性的密文形式。
- 加法同态性：两个 Pedersen 承诺的积等于明文的和的 Pedersen 承诺。假设 $C_1 = g^{m_1} h^{r_1}$ 和 $C_2 = g^{m_2} h^{r_2}$ 是两个 Pedersen 承诺，则有：

$$C_1 \cdot C_2 = g^{m_1} h^{r_1} \cdot g^{m_2} h^{r_2} = g^{m_1+m_2} h^{r_1+r_2}$$

即

$$\mathrm{commit}(m_1, r_1) \cdot \mathrm{commit}(m_2, r_2) = \mathrm{commit}(m_1 + m_2, r_1 + r_2)$$

目录

1 密码学承诺

2 秘密共享
- 基本概念
- Shamir 门限秘密共享
- 其他秘密共享方案

3 安全多方计算

目录

1 密码学承诺

2 秘密共享
- 基本概念
- Shamir 门限秘密共享
- 其他秘密共享方案

3 安全多方计算

秘密共享（Secret Sharing, SS）

- 秘密共享在安全多方计算、分布式系统共识算法等应用中有重要应用。

例

保险柜中存放有 10 个人的共有财产，要从保险柜中取出物品，必须有半数以上的人在场才可取出，半数以下则不行。如何构造锁的设计方案？

例

导弹的发射控制、重要安保场所的通行检验，通常需要多人同时参与才能生效。因此，需要将秘密分给多人掌管，并且由一定掌管秘密的人数同时到场才能恢复秘密。方案如何设计？

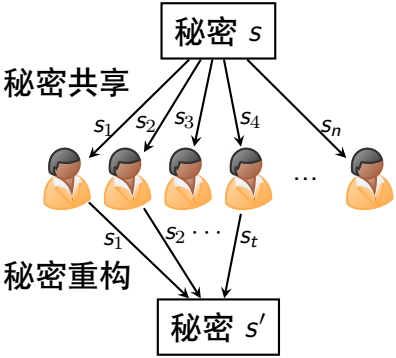
门限秘密共享（Threshold Secret Sharing）

定义（(t,n)-门限秘密共享）

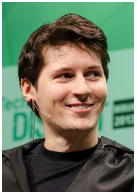
秘密 $s \in \mathbb{Z}$ 被分为 n 个份额（share），由一个参与者持有。如果

- 由 t 个或多于 t 个参与者所持有的部分信息可以重构 s；
- 由少于 t 个参与者所持有的部分信息无法重构 s。

则称该方案为 (t,n)-门限秘密共享，t 称为门限值（Threshold）。



Telegram 与门限秘密分享



- 即时通讯工具 Telegram 的核心竞争力是隐私保护。
- 创始人 Pavel Durov 讲述了 Telegram 如何保存用户消息[1]。
- 所有用户消息均被加密存储，加密密钥被拆分为多份，分散在世界各地，存放在不同法律辖区。
- 这样即使某个政府强制要求获取数据，拿到的也是加密后的乱码，没有完整的密钥无法解密数据。

[1] https://www.bilibili.com/video/BV1FNxLzxET1

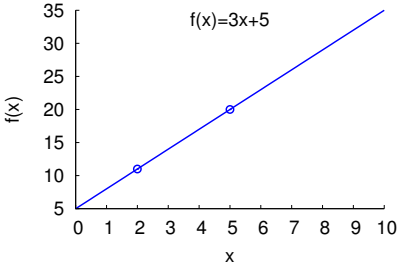
目录

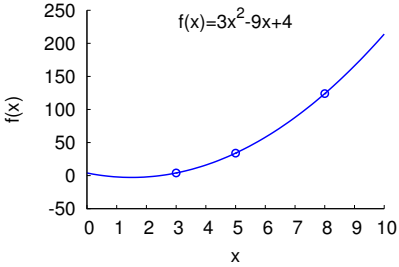
1 密码学承诺

2 秘密共享
- 基本概念
- Shamir 门限秘密共享
- 其他秘密共享方案

3 安全多方计算

Shamir 门限秘密共享的思想





- 一般的，设 $\{(x_1, y_1), \ldots, (x_t, y_t)\}$ 是平面上 t 个不同点，那么存在唯一的不超过 $t-1$ 次多项式
$$f(x) = a_0 + a_1 x + \cdots + a_{t-1} x^{t-1}$$
经过这 t 个点。
- 若把秘密 s 取为 $f(0)$，n 个份额取为 $f(i), i = 1, \ldots, n$，那么利用其中任意 t 个份额便可重构 $f(x)$，从而得到 $s = f(0)$。

Shamir 门限秘密共享

- 设 $\mathrm{GF}(p)$ 为大素数 p 生成的有限域，其中 $p > n$。
- 在 $\mathrm{GF}(p)$ 上构造一个 $t-1$ 次多项式
$$f(x) = a_0 + a_1 x + \cdots + a_{t-1} x^{t-1}$$
其中 $a_0 = s, a_i \in_R \mathbb{Z}_p \backslash \{0\}, i \neq 0$。
- n 个参与者 $u_1, \ldots, u_n$，其中 u_i 持有的份额为 $f(i)$。
- 任意 t 个参与者 $u_{i_1}, u_{i_2}, \ldots, u_{i_t}$ 要重构 s，可以联立方程组
$$\begin{cases} f(i_1) = a_0 + a_1 i_1 + \cdots + a_{t-1} i_1^{t-1} \\ f(i_2) = a_0 + a_1 i_2 + \cdots + a_{t-1} i_2^{t-1} \\ \cdots \\ f(i_t) = a_0 + a_1 i_t + \cdots + a_{t-1} i_t^{t-1} \end{cases}$$
求出 $a_0 = s$。

Shamir 门限秘密共享

- 也可以利用 Lagrange 插值公式进行重构

$$f(x) = \sum_{j=1}^{t} f(i_j) \prod_{l=1, l\neq j} \frac{x - i_l}{i_j - i_l} \pmod{p}$$

- 从而得到

$$s = f(0) = (-1)^{t-1} \sum_{j=1}^{t} f(i_j) \prod_{l=1, l\neq j} \frac{i_l}{i_j - i_l} \pmod{p}$$

Shamir 门限秘密共享的安全性

- 如果有 $t-1$ 个参与者想重构 s，他们可以构造 $t-1$ 个方程，但是有 t 个未知数。
- 由这 $t-1$ 个方程无法得到第 t 个方程的任何信息，第 t 的方程为 $f(0)=s$。
- 可以证明，由 $t-1$ 个份额得不到关于秘密 s 的任何信息。
- 因此 Shamir 门限秘密共享是安全的。

目录

1 密码学承诺

2 秘密共享
- 基本概念
- Shamir 门限秘密共享
- 其他秘密共享方案

3 安全多方计算

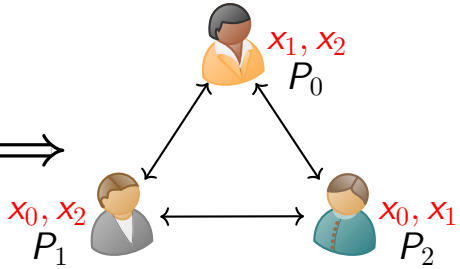
加法秘密共享（Additive Secret Share）

- 加法秘密共享是一种更简单的秘密共享方案，可以看作是门限秘密共享的特例：(n, n)-门限秘密共享。
- 假设要把秘密 $s \in \mathbb{Z}$ 共享给 n 个人，要求 n 个人都同意后，才可以由秘密份额恢复出原始秘密 s。
- 任意选择 $n-1$ 个随机数 $x_i \in \mathbb{Z}, i = 1, \ldots, n-1$，第 n 个随机数 x_n 为
$$x_n = x_1 \oplus \cdots \oplus x_{n-1} \oplus s$$
- 将 $x_1, \ldots, x_n$ 分发给 n 个参与者，完成秘密共享。
- 当 n 个参与者都同意恢复秘密 s 时，每个人交出 x_i 然后计算
$$s = x_1 \oplus \cdots \oplus x_n$$
- 异或运算 $\oplus$ 也可以变为加法运算 $+$。

三方复制秘密共享 Three-Party Replicated Secret Sharing

- 要求参与方数量 $n \geq 3$，以三方为例，第 i 个参与方记为 P_i，$i = 0, 1, 2$。
- 将秘密 x 以加法秘密共享的方式拆分为三个份额 x_0, x_1, x_2，满足 $x = x_0 + x_1 + x_2$。
- 第 i 个参与方 P_i 持有 $x_{(i-1) \bmod 3}, x_{(i+1) \bmod 3}$，简记为 x_{i-1}, x_{i+1}
- 特点：任意两方就可以重构秘密 x。

$$x \Longrightarrow (x_0, x_1, x_2) \Longrightarrow$$



目录

1 密码学承诺

2 秘密共享

3 安全多方计算
- 基本概念
- 基于秘密共享的 MPC
- 基于混淆电路的 MPC

目录

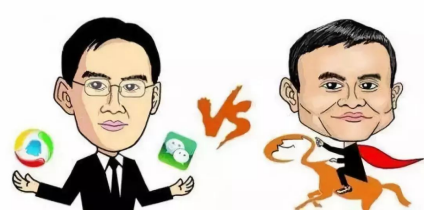
1 密码学承诺

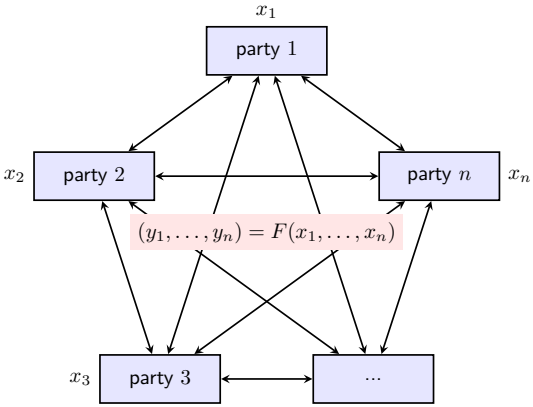
2 秘密共享

3 安全多方计算
- 基本概念
- 基于秘密共享的 MPC
- 基于混淆电路的 MPC

安全多方计算（Secure Multi-Party Computation, MPC）

- MPC 始于姚期智于 1982 年提出的百万富翁问题：两个富翁在不泄漏个人财产具体数额的前提下，如何比较谁更富有？
- 一般情况：n 个参与方在不泄露各自隐私数据 x_i 的前提下计算函数 $(y_1, \ldots, y_n) = F(x_1, \ldots, x_n)$。





目录

1 密码学承诺

2 秘密共享

3 安全多方计算
- 基本概念
- 基于秘密共享的 MPC
- 基于混淆电路的 MPC

基于秘密共享的安全多方计算：加法协议

- 问题：两方各自持有隐私数据 x 和 y，计算 $z = x + y$。

Alice(x)		**Bob**(y)
$(x_1, x_2) \leftarrow x$	发送份额 x_2 $\longrightarrow$	x_2
y_1	$\longleftarrow$ 发送份额 y_1	$(y_1, y_2) \leftarrow y$
$z_1 = x_1 + y_1$		$z_2 = x_2 + y_2$
	发送份额 z_1 $\longrightarrow$	z_1
z_2	$\longleftarrow$ 发送份额 z_2	
$z = z_1 + z_2$		$z = z_1 + z_2$

- 由 z_1, z_2 得到 z 的过程也叫作公开 z。

基于秘密共享的安全多方计算：乘法协议

- 问题：两方各自持有隐私数据 x 和 y，计算 $z = xy$。
- 乘法不再具有线性性
$$(x_1 + x_2)(y_1 + y_2) = x_1y_1 + x_2y_2 + x_1y_2 + x_2y_1$$
- Beaver 三元组：三个随机数 (a, b, c) 且满足 $c = ab$。
- Setup: 双方持有 Beaver 三元组的加法秘密共享：Alice 持有 (a_1, b_1, c_1)，Bob 持有 (a_2, b_2, c_2)，双方都不知道 (a, b, c)。
- 注意到
$$\begin{aligned} xy &= (x + a - a)(y + b - b) \\ &= (x + a)(y + b) - (x + a)b - a(y + b) + ab \end{aligned}$$
- 令 $A \triangleq x + a, B \triangleq y + b$，则
$$xy = AB + Ab + aB + c$$

基于秘密共享的安全多方计算：乘法协议

- **注意到**：即使公开 A, B，也不会泄露隐私数据 x, y。
- Alice 和 Bob 执行两次加法协议，公开 A, B，则

$$z_1 = AB + Ab_1 + a_1 B + c_1$$
$$z_2 = Ab_2 + a_2 B + c_2$$

Alice(x)		**Bob**(y)
a_1, b_1, c_1	秘密共享 $\longleftrightarrow$	a_2, b_2, c_2
x_1, y_1	秘密共享 $\longleftrightarrow$	x_2, y_2
A, B	公开 A, B $\longleftrightarrow$	A, B
计算 z_1		计算 z_2
$z = z_1 + z_2$	公开 z $\longleftrightarrow$	$z = z_1 + z_2$

基于三方复制秘密共享的安全多方计算

- 问题：两方各自持有隐私数据 x 和 y，计算 $z = xy$。
- 产生 x 和 y 的加法秘密共享 x_i 和 y_i，$i = 0, 1, 2$。
- 三个计算方 P_0, P_1, P_2，其中 P_i 持有 $x_{i-1}, x_{i+1}, y_{i-1}, y_{i+1}$。
- P_i 计算

$$z_i = x_{i-1}y_{i+1} + x_{i+1}y_{i-1} + x_{i+1}y_{i+1}$$

- 则有

$$\begin{aligned} z_0 + z_1 + z_2 &= (x_2y_1 + x_1y_2 + x_1y_1) + (x_0y_2 + x_2y_0 + x_2y_2) \\ &\quad + (x_1y_0 + x_0y_1 + x_0y_0) \\ &= x_0(y_0 + y_1 + y_2) + x_1(y_0 + y_1 + y_2) \\ &\quad + x_2(y_0 + y_1 + y_2) \\ &= (x_0 + x_1 + x_2)y = xy = z \end{aligned}$$

基于三方复制秘密共享的安全多方计算

$\mathbf{P_0}$		$\mathbf{P_1}$		$\mathbf{P_2}$
x_1, x_2, y_1, y_2		x_0, x_2, y_0, y_2		x_0, x_1, y_0, y_1
计算z_0		计算z_1		计算z_2
	$\xrightarrow{z_0}$	z_0	$\xrightarrow{z_1}$	z_1
z_2	$\xleftarrow{z_2}$			

- 进行乘法计算时每个参与方 P_i 只需本地计算，产生 z_i。
- 然后 P_i 向 P_{i+1} 发送 z_i，完成复制秘密共享。
- 通信复杂度为常量，适合进行向量内积和矩阵运算。

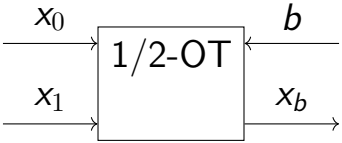
目录

1 密码学承诺

2 秘密共享

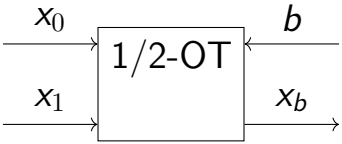
3 安全多方计算
- 基本概念
- 基于秘密共享的 MPC
- 基于混淆电路的 MPC

不经意传输（Oblivious Transfer, OT）



- OT 是安全计算的基本原语，最简单的 OT 是 $1/2$-OT。
- 两方：发送方和接收方。
- 发送方持有两个秘密串 $x_0, x_1 \in \{0,1\}^n$，接收方持有选择位 $b \in \{0,1\}$。
- 接收方发送 b 给 OT，发送方发送 x_0, x_1 给 OT；
- 接收方收到 x_b，发送方收到 $\perp$。
- $1/2$-OT 可以推广到 $1/n$-OT。

一种基于公钥密码学的 1/2-OT 实现



- 接收方生成公私钥 (sk, pk)，随机数 pk'
  - 如果 $b = 0$，发送 (pk, pk')；
  - 如果 $b = 1$，发送 (pk', pk)；
- 发送方收到 (pk_0, pk_1)，发送
$$(e_0, e_1) = (\mathrm{E}_{pk_0}(x_0), \mathrm{E}_{pk_1}(x_1))$$
- 接收方收到 (e_0, e_1)，用 sk 解密 e_b。

姚氏混淆电路（Yao's Garbled Circuit, GC）

x	y	$F(x,y)$
00	00	$F(00,00)$
00	01	$F(00,01)$
00	10	$F(00,10)$
00	11	$F(00,11)$
01	00	$F(01,00)$
01	01	$F(01,01)$
01	10	$F(01,10)$
01	11	$F(01,11)$
$\vdots$	$\vdots$	$\vdots$

- 函数 $F(x,y)$ 可以看作一张查找表，输入隐私数据 x,y，输出结果 $F(x,y)$。
- 若 $x \in X, y \in Y$，则该表有 $|X| \cdot |Y|$ 行，记 (x,y) 行的函数值为 $T_{x,y} \triangleq F(x,y)$。
- P1: 为每个 x 和 y 分别生成随机密钥 $k_x, k_y \in \{0,1\}^n$，对表加密 $\mathrm{E}_{k_x,k_y}(T_{x,y})$，发送加密后的表给 P2。
- P2: 为了得到 $T_{x,y}$，需要 k_x, k_y，其中 k_x 由 P1 直接发送给 P2，而 k_y 通过执行 $1/|Y|$-OT 实现。
- 优点：两方只进行一轮交互。

Point-and-Permute

- P2 收到加密后的表，使用密钥 k_x, k_y 只能正确解密其中一行。
- 如果 P2 事先不知道应该解密表的哪一行，那么需要尝试解密所有行，效率低。
- Point-and-Permute：在密钥中编码行号信息，即用 k_x 的前 $\log|X|$ 比特和 k_y 的前 $\log|Y|$ 比特指定需解密的行号。
- P2 收到密钥 k_x, k_y 后，可以确定出需要解密的行。

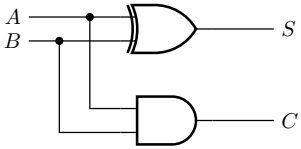
k_x	k_y	$\mathrm{E}_{k_x,k_y}(T_{x,y})$
$\bullet*$	$\bullet*$	$\mathrm{E}_{\bullet*,\bullet*}(T_{x,y})$
$\bullet*$	$\bullet*$	$\mathrm{E}_{\bullet*,\bullet*}(T_{x,y})$
$\bullet*$	$\bullet*$	$\mathrm{E}_{\bullet*,\bullet*}(T_{x,y})$
$\bullet*$	$\bullet*$	$\mathrm{E}_{\bullet*,\bullet*}(T_{x,y})$

$\xrightarrow{\text{排序}}$

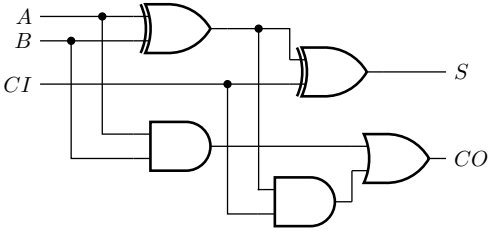
	$\mathrm{E}_{k_x,k_y}(T_{x,y})$
$\bullet\bullet$	$\mathrm{E}_{\bullet*,\bullet*}(T_{x,y})$
$\bullet\bullet$	$\mathrm{E}_{\bullet*,\bullet*}(T_{x,y})$
$\bullet\bullet$	$\mathrm{E}_{\bullet*,\bullet*}(T_{x,y})$
$\bullet\bullet$	$\mathrm{E}_{\bullet*,\bullet*}(T_{x,y})$

混淆电路

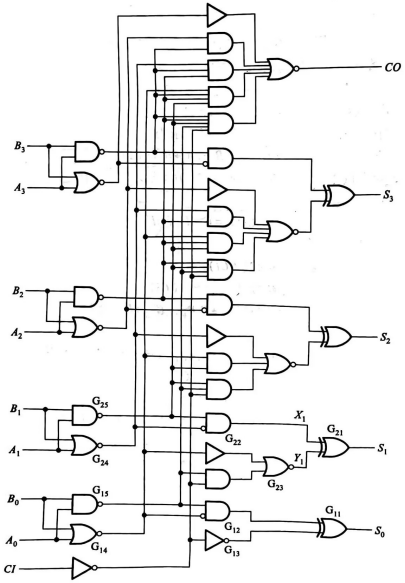
- 如果 $|X|, |Y|$ 很大，那么表很大。
- 任何计算都可以用门电路实现。



半加器

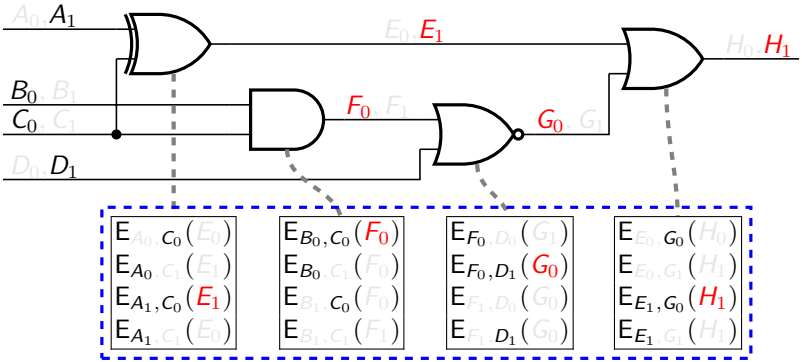


全加器



四位加法器

混淆电路



P1 创建混淆电路：

- 为每根线随机选择标签 W_0, W_1
- 混淆每个门的真值表
- 加密后的门构成混淆电路

P2 计算混淆电路：

- 获取所有输入线的标签
- 依次解密每个门的输出标签
- 最后将输出线的标签映射为真值，完成电路计算

两种安全多方计算比较

- 混淆电路（GC）：
  - 计算轮数为有限次
  - 适合在高延时网络中进行 MPC 计算
- 秘密共享（SS）：
  - 虽然需要多次交互，但总体内存、计算等开销小于基于 GC 的 MPC。
  - 适合在低延时网络中进行 MPC 计算。

小结

1. 密码学承诺
2. 秘密共享
3. 安全多方计算